

# src\case\_studies\D\_mass\pd\_controller.py

Generated: 2026-02-20 21:11:26

pd\_controller.py

src\case\_studies\D\_mass\pd\_controller.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from .. import common
from . import params as P
from ..control.pd import PD
from ..control import utils_design

class MassControllerPD(common.ControllerBase):
    def __init__(self, chapter8=False):
        # system parameters
        b0 = P.tf_num[-1]
        a1, a0 = P.tf_den[-2:]

        if chapter8:
            # tuning parameters
            tr = 2.0 # original
            tr = 1.65 # tuned to barely saturate on first step only
            zeta = 0.707

            # desired characteristic equation parameters
            wn = utils_design.get_natural_frequency(tr, zeta)
            alpha1 = 2.0 * zeta * wn
            alpha0 = wn**2

        else: # chapter 7
            # tuning parameters
            p1 = -1.0
            p2 = -1.5

            # desired characteristic equation parameters
            # des_CE: s^2 + alpha1*s + alpha0 = s^2 + (a1 + b0*kd)s + (a0 + b0*kp)
            des_CE = np.poly([p1, p2])
            alpha1, alpha0 = des_CE[-2:]

        # find gains
        kp = (alpha0 - a0) / b0
        kd = (alpha1 - a1) / b0
        print(f"kp = {:.2f}, {kd = {:.2f}}")

        self.z_pd = PD(kp, kd)
        self.F_eq = 0.0
        self.chapter8 = chapter8

    def update_with_state(self, r, x):
        # unpack references and states
        z_ref = r[0]
        z = x[0]
        zdot = x[1]
```

```
# z (modified) PD
F_tilde = self.z_pd.update_modified(z_ref, z, zdot)
u = np.array([F_tilde + self.F_eq])

if self.chapter8: # don't change results from chapter 7 PD
    u = self.saturate(u, u_max=P.force_max)
return u
```